

D1 (LDS计数)

CF2043D1

- 题意：定义一个序列是合法的，当且仅当存在一种方法给每个下标 i 涂成蓝色或红色，使得对于任意一对满足 $i < j$ 且 $b_i > b_j$ 的下标， i 和 j 这两个位置的颜色都不相同。现在给定一个长度为 N 的数组 A ，计算有多少个子序列是合法的，结果对 $1e9 + 7$ 取模
- 范围： $N \in [1, 300]$, $A_i \in [1, n]$
- 关键思考：
 - 本题为计数类题目，常用思考方式为：暴力 $\rightarrow DP$ || 组合数学，常用优化手段为：前缀和、双指针、数学
 - 首先分析题目性质：不难发现，一个序列 S 是合法的等价于 S 的最长严格下降子序列 LDS 的长度 ≤ 2 。因为若 $b_i > b_j$, $b_j > b_k$, 就一定有 $b_i > b_k$ 。这样由抽屉原理， i, j, k 三个下标至少两个相同
 - 拿到计数题，我们首先要思考要从正面计数还是反面计数
 - 如果从反面计数，考虑用合法数 = 总数 - 不合法数，就会遇到非常多的去重/算漏难题。例如 $[3, 2, 4, 2, 1]$ ，无论如何定义，都无法很方便地计算不合法数，原因在于不合法是一个局部性质，一个序列的不合法性可以由很多个局部不合法导致。相对应的，合法性是个整体性质，研究整体性质就能避免很多去重问题
 - 先来思考简单情况：如果要求 LDS 的长度 ≤ 1 ，此时该如何计算呢？
 - 不难想到，这等价于 IS 计数问题，即求解有多少个上升子序列，这是经典的树状数组优化 DP 问题
 - 定义 $dp[i][x] :=$ 考虑前 i 个元素，以值 x 为结尾的上升子序列个数
 - 在处理第 i 个元素时，有两种转移方程：
 - 不选 a_i : $dp[i] = dp[i - 1]$
 - 选 a_i : $dp[i][a[i]] += \sum_{x=1}^{a[i]} dp[i - 1][x]$
 - 思考：我们为什么要把 IS 序列的最后一个元素（最大元素）作为一维状态呢？
 - 其实，这是在保存**最有潜力破坏IS**的值
 - 考虑 a_i 能否被加入到某个 IS 结尾，关键在于 a_i 是否大于等于该 IS 的最后一个元素，因此我们就需要把最后一个元素作为状态来存起来，才能实现正确的转移
 - 回到本题，现在放宽条件，要求 LDS 的长度 ≤ 2 ，这又该如何设计状态呢？
 - 让我们从一个例子入手： $[3, 2, 5, 1, 4, 3]$
 - 很自然的想法是，对上面的 IS 计数问题稍作修改：
 - 定义 $dp[i][x] :=$ 考虑前 i 个元素，满足 $\max \{S_u | \exists v < u, S_v > S_u\} = x$ 的子序列个数
 - 这样在转移时，由于 x 是**最有潜力的**，任何 $< x$ 的数都无法加入到该子序列的末尾，任何 $\geq x$ 的数都能加入到该子序列末尾
 - 但问题也随之而来，对于 $\geq x$ 的数，我们会发现无法进行转移，因为把 a_i 添加到子序列末尾后，我们并不知道子序列中是否有某个元素 $> a_i$ ，因此就无法确定究竟是转移到 $dp[i][x]$ 还是 $dp[i][a[i]]$ 了
 - 解决办法很简单，当我们发现无法实现转移的时候，就需要尝试定义些新的辅助状态。上面的问题在于

无法判断子序列中是否有某个元素 a_i ，那么我们就可以对症下药，把子序列的最大值也作为一维状态，这样根据 max 与 a_i 的关系我们就知道如何进行转移了

○ 于是有了以下做法：

- 定义 $dp[i][mx][x]$:= 考虑前 i 个元素，满足 $max\{S_u | \exists v < u, S_v > S_u\} = x$ 且 $max(S) = mx$ 的子序列个数

- 转移方程为：

- 不选 a_i : $dp[i] = dp[i - 1]$

- 选 a_i ，且添加到已有子序列末尾：此时必须满足 $x \leq a_i$ 才能进行转移

- $dp[i][mx][a[i]] += \sum_{x \leq a_i} dp[i - 1][mx][x], \text{ if } x < mx$

- $dp[i][a[i]][x] += \sum_{x < mx} dp[i - 1][mx][x], \text{ if } x \geq mx$

- 选 a_i ， a_i 单独作为长度为1的新子序列出现：

- $dp[i][a[i]][0] = 1$

- 最终答案为：

- $1 + \sum_{mx=1}^n \sum_{x=0}^{mx-1} dp[n][mx][x]$